ROCHESTER HILLS PUBLIC LIBRARY · IT INTEGRATION & SECURITY

Autolend Holds-Locker Kiosk — PAPI Integration & Device-Access Security

Prepared by Derek Brown, Director of IT Updated July 1, 2026

🔒 IT INTERNAL

Oakland Township is moving ahead with the ILS **AutoLend** holds locker — **on the condition that ILS delivers the Polaris integration over PAPI** (the modern Polaris API), not the older SIP2 protocol. With the purchase decision settled, RHPL's job is the technical part: standing up a clean PAPI integration to our Clarivate-hosted Polaris, and securing how staff interact with the device. This document covers three things — **why PAPI**, **what the integration looks like**, and **our security requirements for staff access to the unit**. We are confident this is achievable and are prepared to do the legwork with ILS and Clarivate.

Why PAPI, not SIP2

- **Security.** PAPI is a modern web API that runs over **HTTPS and encrypts natively**, with least-privilege, per-purpose credentials. SIP2 is an older protocol that sends the patron's barcode + PIN **in the clear by default**; on a Clarivate-hosted instance it can only be secured by bolting on a separate `stunnel` tunnel *and* a dedicated static, allow-listed IP. PAPI needs none of that scaffolding — the encryption is built in.
- **Seamless fit.** RHPL already runs several third-party vendors against our Clarivate-hosted Polaris over PAPI. It is a well-worn, supported path on our end — not a special case.
- **Simplicity — and no dedicated IP.** PAPI removes the biggest moving parts of the old SIP2 plan: the stunnel tunnel *and* the mandatory static carrier IP for the SIP allow-list. PAPI is **not IP-restricted** — access is granted by an RHPL-built API key (plus a staff logon for protected calls), not by source IP — so it needs no static IP, no allow-list, and no VPN. We know this firsthand: RHPL already runs PAPI this way in production (see the callout below).

This is exactly why Oakland Township's decision is conditioned on PAPI: it is both the safer path for patron credentials and the cleaner fit with how RHPL already operates.

What the PAPI integration looks like

We walked the full patron journey and checked *each* required call against **Polaris's own published API documentation** for our version (**Polaris 8.1**). Every call this needs already exists, and the one most central to the catalog display we have **already run on our own live catalog**.

Step in the transaction	What it does	Polaris PAPI method	Access	Exists in 8.1?
1. Patron sign-in	Validate the patron's card + PIN and read account status/blocks before anything is lent	`AuthenticatePatron`	Public (patron)	**Yes — High**
2. Find the ready hold	Look up the patron's holds ready for pickup and match to what's in the locker	`PatronHoldRequestsGet` (+ `HoldRequestCreate` / `Reply` , `HoldPickupAreaID`)	Mixed	**Methods exist** Locker hold workflow to confirm hands-on with the vendor
3. Check out (charge)	Charge the retrieved item to the patron (auto-renews if they already have it)	`ItemCheckoutPost` (Polaris 7.4)	Public (patron)	**Yes — High**
4. Check in (discharge / "smart return")	Record a returned item as checked in and available again	`ItemCheckinPost` (Polaris 7.7)	Protected (staff)	**Yes — High**
5. Book data for browsing	Pull title, author, summary and ISBN to display the item on the machine's screen	`BibGetByType` (v2 barcode lookup)	Public	**Yes — Very High** Already run against RHPL's live catalog
6. Cover artwork	Show the book's jacket image on the browse screen	ISBN (from step 5) → cover-image service	—	**Achievable** Bib/summary from PAPI; the jacket image is normally fetched by ISBN from a cover-art provider

What it takes to stand up (provisioning) — and RHPL controls it

This is the part we self-service. Access to PAPI on our Clarivate-hosted instance is granted by credentials RHPL builds in **Polaris Admin**, hitting the public PAPI address (`https://catalog.rhpl.org/PAPIService`) over HTTPS — **no dedicated IP, no allow-list, and no network path for Clarivate to open**:

- **A PAPI API key** (an Access ID + secret) built in Polaris Admin, scoped to the methods the device needs. Requests are signed with it (PWS / HMAC) — that is the "key + passcode" that authorizes API access.
- **A dedicated, least-privilege staff logon for the device.** Check-in (discharge) is a *protected* method, so the device authenticates as a scoped Polaris staff user (`AuthenticateStaffUser` → access token) — its own circulation-only service account, not a person's login. Patron sign-in, *check-out*, and the book-data lookup are public patron/API-key methods (the same pattern our eResources service already uses), so the staff logon is really only needed for discharge.
- **A live go-live test on our own instance** before we accept it. ILS asks for this too — Bill: *"We will test Holds pickup with your team to ensure we get the results we expect. Every site's holds can be different."* We extend that to charge and discharge as well.

> **We already run PAPI exactly this way — proof it needs no dedicated IP.** Two RHPL production services authenticate to our hosted Polaris with nothing but an RHPL-built PAPI key (Access ID `localpull`) over public HTTPS: our **student-barcode update pipeline** performs *protected, staff-authenticated patron writes* from the localai server, and our self-hosted **eResources** service authenticates patrons from **Google Cloud Run** — whose egress IP isn't fixed at all. Neither uses a static IP, an allow-list, or a VPN. If AutoLend runs on PAPI, its access model is the same one we already operate.

> **Confidence: high — and we are prepared to help make it happen.** Every step maps to a working, documented Polaris call, all present in the 8.1 release we run; "can Polaris do this over PAPI?" is settled: **yes**. RHPL is ready to work directly with ILS on the setup and coordinate the PAPI provisioning with Clarivate support. Our goal is a clean, secure integration and a good experience for patrons, and we are glad to do the technical legwork.

> **The one honest caveat: RHPL would be ILS's first production PAPI-transactional site.** By ILS's own account these calls are tested but not yet running in production anywhere — Bill: *"We have tested the PAPI patron, hold, and most other calls previously… we have never had anyone in production request charge or discharge. It's a bit like NCIP — we support it, but have no production sites using it so far."* The platform risk is gone; the remaining risk is the vendor's *implementation* in production — which is exactly why the language in the quote names every transaction and ties acceptance to a successful end-to-end test.

Security: how staff interact with the device

Setting aside the Polaris connection, the other half of this is **how staff touch the unit**. There are three surfaces — the on-device Administration screen, the staff web dashboard used from the library, and the vendor's remote-support tool. Below is what ILS offers for each (in Bill's own words) and the method we want.

1. At the machine — the Administration screen

ILS's on-device admin is a local card/number that only the box checks, scoped by role:

BILL MCCLENDON, CTO — INTERNATIONAL LIBRARY SERVICES (VERBATIM)

> *Administration screen access is very simple on the device, if you have a card or number assigned, you get a screen. The features available on the screen are configurable, so if you are a courier and you only need to add material or remove material… you are limited to what is assigned. But the person there at that screen cannot do anything but what we allow… They can't modify records, gain additional access, etc.*

> *The ID/number used to access the Administration screen can be a card that has no connection to the ILS at all. It can be a dummy card (or cards), an employment ID, a badge number, whatever number you want… It's strictly local. Most sites grab 1 (or more) patron cards not assigned or used, write "Admin" on the card, then stick it in a drawer or on a lanyard… Admin access is more secure that way, as there is nothing to protect.*

Where we differ — and the method we want. The "one Admin card in a drawer" model is low-risk in isolation (local-only, can't modify records), but a *shared* card gives **no accountability and no clean offboarding**. Our requirement: **each staff member who services the unit gets their own individual Admin ID/PIN**, so the device log shows *who* loaded or serviced it, and we can revoke one person without reissuing everyone. It stays a quick tap — just per-person, not a communal card. **Ask ILS to confirm** the device supports multiple named Admin IDs and keeps a local activity log of which ID did what. (Bill also notes an optional path if we want more: *"we have an OAuth approach that we did for Academics, so we have implemented MFA… If you really want to go down that path for just the Administration screen, we can discuss that further."*)

2. From the library — the staff web dashboard

The bigger surface is the web dashboard staff use remotely (create users; view machine status, inventory, and check-in/out history). Bill was candid that it is unhardened by default, and laid out the options:

BILL MCCLENDON (VERBATIM)

> *The web traffic for staff is served from the device and defaults to HTTP. Now, we do not store nor display ANY personal information AT ALL. But if you want to secure that traffic (HTTPS), you would need to get a certificate for the device… We would then configure, install, and test your SSL… Almost none of our current sites encrypt the staff web pages… but you seem a thorough sort to me.*

> *We can limit those to allow only certain LAN IP's as well… if you wanted to, you could setup a proxy to the staff web pages, so you could have staff connect on your side, authenticate, vet/challenge/MFA, whatever, then allow the connection through. That gives you total control within your current security framework with no change other than the proxy page they use.*

The method we want: Bill's proxy option. Put the dashboard behind **our own login-proxy** — the same Google sign-in + YubiKey (FIDO2) pattern we run for the internal reports portal — and **IP-restrict the device so only our proxy can reach it**, over HTTPS with our own certificate. That gives us, in one move:

- **Encrypted (HTTPS), never plain HTTP** — using our certificate.
- **No public opening** — the dashboard is never sitting exposed at the device's address; only our authenticated proxy can reach it.
- **Individual named accounts + MFA** at the front door — no shared logins, no default passwords, lockout on repeated failures, all enforced by our identity stack.

The gap to close. Our proxy is a strong front door, but once a user is through it the *vendor's* app still governs what they can do. So confirm with ILS, in writing, the dashboard's own account model behind the proxy: **individual named accounts, least-privilege roles** (especially limiting who can "create users"), **failed-login lockout, and an audit log** of who changed what.

3. Vendor remote support — Splashtop

ILS supports the unit through a remote-access tool and offers RHPL IT a free account on it:

BILL MCCLENDON (VERBATIM)

> *It's called Splashtop… this product is a full VPN approach, so any/all traffic over the connection — from before login, to after disconnect — is encrypted… to connect, you must use either an access code or account and password… you can configure your account to use their MFA approach, but they don't support your approach in their setup.*

The method we want. Splashtop is cloud-brokered and **outbound-only** (no inbound port opened on the device), which is acceptable. **Take the free IT account for visibility and audit**, and **enforce Splashtop's**

own MFA (we can't use our identity provider here). Bound and document it: this is a vendor control plane that can fully drive the Windows box, so we keep the device's Polaris credential least-privilege and log its egress. This is the one exposure we *bound* rather than eliminate — inherent to any vendor-managed appliance, and we name it rather than hide it.

> **An honest word on the vendor.** ILS said *yes* to every hardening measure we raised (HTTPS with our cert, IP-restriction, our own login-proxy, per-role scoping, MFA options, read-only catalog access) and **volunteered** that the staff page defaults to HTTP rather than letting us discover it. They were also candid that *"almost none of our sites encrypt the staff web pages"* and no one has asked for challenge/response on the admin screen — so the hardening we want is achievable but somewhat new territory for them. Our posture is **"verify, don't distrust."**

What we need to close this out

A short list of the confirmations and setup steps that get us from "agreed in principle" to "live and secure":

- **ILS — commit the PAPI scope in writing:** patron authentication, check-out, check-in, hold pickup, and catalog content all over PAPI, with no SIP2 for any patron/material transaction (the explicit quote language RHPL supplied).
- **ILS — individual Admin IDs** for the on-device screen (not one shared card) + a local activity log.
- **ILS — dashboard account model** behind our proxy: named accounts, least-privilege roles, lockout, audit log; HTTPS with our cert; IP-restricted to our proxy.
- **ILS — end-to-end PAPI test** (auth + checkout + checkin + holds) on our Clarivate-hosted instance before acceptance/final payment.
- **RHPL — build the PAPI access in Polaris Admin** (self-serviced): an API key + a least-privilege circulation service account for the device. No dedicated IP is needed and this is self-serviced in Polaris Admin — we already run other PAPI services this way; the only thing that might involve Clarivate is registering a brand-new PAPI application, not the access itself.
- **RHPL — stand up the login-proxy + device connectivity** and run the go-live test with ILS.

Appendix — vendor correspondence timeline (June 16–30, 2026)

A running summary of the ILS (Bill McClendon) email thread — roughly a dozen exchanges over two weeks. The key quotes are pulled inline above; the full verbatim Q&A is retained in RHPL's working file.

- **June 16–17 — first contact & the thorough reply.** Bill confirmed he is the right technical contact and sent detailed answers plus the (confidential) staff manual, establishing: SIP2 for circulation/holds and PAPI read-only for catalog/cover-art at the time; holds route to the locker like any pickup branch and fire the **standard Polaris notice**; offline mode off → a configurable **"Out of Service"** screen with no patron

data stored; the staff interface is fully web-based on a vendor-managed **Windows 10 IoT** box; strong accessibility.

- **June 18 — the hardening reply.** stunnel available for SIP2; accepts an Ethernet/Wi-Fi WAN and endorsed our Peplink approach; staff dashboard defaults to HTTP (no PII) and can be HTTPS with our cert, IP-restricted, or fronted by our own auth/MFA proxy; on-device admin is a local card/number (can be a dummy card), role-scoped; RMM is **Splashtop** (cloud-brokered/outbound, own MFA, free IT account); catalog access read/search only.
- **June 19–22 — the barcode→record (BibGetByType v2) exchange.** Bill was skeptical the PAPI barcode→bib lookup was possible; RHPL demonstrated it working on our live catalog (shipped in Polaris 7.6; appears under the Swagger "v2" definition; in use at SILS and a Phoenix locker system).
- **June 22–23 — references offered.** A long-time site (10 units since Jan 2023, now hosted) and Lake Forest (part of the CCS consortium, which handled that site's networking/security).
- **June 30 — the PAPI pivot, then walk-back.** Bill first stated ILS does PAPI "within all our ILS products including the AutoLend," then under questioning clarified: *"Yes, we have used SIP2 for all material transactions, but we can switch to PAPI whenever asked — we only need to test before production."* On the transactional calls: tested previously, but *"no production sites using it so far"* — the basis for the first-production-site note above.
